

1. BIG PICTURE Y FLUJO DE TRABAJO

JERARQUÍA Y DEFINICIONES

- **Inteligencia Artificial (AI):** Agentes racionales que mapean perceptos a acciones.
- **Machine Learning (ML):** Aprendizaje estadístico sin programación explícita de reglas.
- **Deep Learning (DL):** Redes neuronales (ANN). A diferencia del ML clásico, no satura su rendimiento al escalar los datos.
- **Stack Tecnológico:** DL requiere alta paralelización (multiplicaciones matriciales). Algoritmo (PyTorch) → Grafo Computacional → API (CUDA) → Hardware (GPU).

TAXONOMÍA DEL APRENDIZAJE

- **Supervisado:** Dataset etiquetado $\mathcal{D} = \{(x_i, y_i)\}$. Busca $f(x) \approx y$. Clasificación (salida discreta) y Regresión (salida continua).
- **No Supervisado:** Dataset sin etiquetas $\mathcal{D} = \{x_i\}$. Extrae estructura latente. Clustering y Reducción de Dimensionalidad (PCA).
- **Refuerzo (RL):** Optimiza política $\pi(a|s)$ iterando Agente → Entorno → Reward R.

ML WORKFLOW (7 PASOS)

- 1) **Definición:** Establecer meta y requerimientos físicos.
- 2) **Colección y Reducción:** Filtrado de ruido, homogeneización.
- 3) **Data Split:** Partición en Train, Validation y Test.
- 4) **Selección de Modelo:** Inducción del sesgo algorítmico adecuado.
- 5) **Entrenamiento y Evaluación:** Optimización de pesos.
- 6) **Tuning:** Ajuste de hiperparámetros con el set de validación.
- 7) **Análisis Final:** Evaluación de generalización en Test Set.

II. MAXIMUM LIKELIHOOD ESTIMATION (MLE)

El estimador analítico Ordinary Least Squares (OLS) minimiza $\sum (y - \hat{y})^2$. Limitaciones:

- 1) **Alta dimensión:** Si $d \gg N$, existen infinitas soluciones equivalentes (matriz singular).
- 2) **Multicolinealidad:** Variables correlacionadas inflan la varianza de los coeficientes.
- 3) **Overfitting:** Síntoma: Coeficientes w enormes y erráticos.

MÁXIMA VEROSIMILITUD (MLE)

Asumiendo datos i.i.d. (Independent and Identically Distributed), la versoimilitud $\mathcal{L}$ es la probabilidad conjunta de las observaciones dado el parámetro θ . Para maximizar la verosimilitud, se minimiza la *Negative Log-Likelihood* (NLL):

$$\mathcal{L}(\theta) = \prod_{i=1}^N p(y_i | \mathbf{x}_i; \theta) \implies \theta_{MLE}^* = \arg \min_{\theta} \sum_{i=1}^N -\ln p(y_i | \mathbf{x}_i; \theta)$$

Regresión lineal: if ruido Gaussiano $\epsilon \sim \mathcal{N}(0, \sigma^2)$: $\arg \max \mathcal{L} \equiv \text{MSE}$.

Regresión logística $y \in \{0, 1\}$: if $y_i \sim \text{Bernoulli}(\hat{y}_i = \sigma(w^T \mathbf{x}_i))$: $\arg \max \mathcal{L} \equiv \arg \min \text{BCE}$

III. LA NEURONA Y OPTIMIZACIÓN

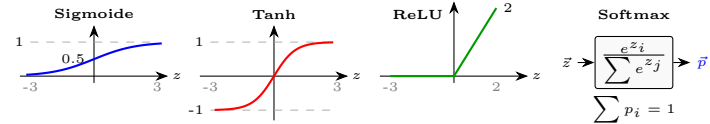
ARQUITECTURA DE LA NEURONA

Mapeo afín seguido de activación no lineal. Sin $f(z)$, la red colapsa a una regresión lineal.

$$z_j = \sum_i w_{ij} x_i + b_j = w_j^T \mathbf{x} + b_j \implies a_j = f(z_j)$$

FUNCIONES DE ACTIVACIÓN $f(z)$

- **Sigmoido:** $\sigma(z) = (1 + e^{-z})^{-1} \in (0, 1)$, $\sigma' = \sigma(1 - \sigma)$. Defecto: $|z| \rightarrow \infty \Rightarrow \sigma' \rightarrow 0$, causando *Vanishing Gradient*. No está centrada en cero (oscilaciones en SGD).
- **Tanh:** $\tanh(z) \in (-1, 1)$, $\tanh'(z) = 1 - \tanh^2(z)$. Centrada en 0 (convergencia más rápida), pero mantiene saturación hiperbólica.
- **ReLU:** $\max(0, z)$. Estándar DL. Computación trivial, previene desvanecimiento para $z > 0$, induce *sparsity*. Defecto: Dead ReLU si cae en $z \leq 0 \Rightarrow \nabla = 0$.
- **Softmax:** Generalización para K clases. Convierte un vector de *logits* en una distribución de probabilidad normalizada $\sum p_j = 1$.



FUNCIONES DE PÉRDIDA ($\mathcal{L}$)

Definen la topología de la superficie de error a optimizar.

- **MSE:** $\frac{1}{N} \sum (y_i - \hat{y}_i)^2$. Regresión. Sensible a outliers.
- **MAE:** $\frac{1}{N} \sum |y_i - \hat{y}_i|$. Regresión. Geometría robusta a outliers.
- **MRE (Error Relativo):** $\frac{1}{N} \sum \frac{|y_i - \hat{y}_i|}{|y_i|}$. Adimensional.
- **BCE:** $-\frac{1}{N} \sum [y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i)]$. Clasificación binaria.
- **CCE:** $-\sum y_c \ln(\hat{y}_c)$. Clasificación multiclase con One-Hot Encoding.

STOCHASTIC GRADIENT DESCENT (SGD) Y BACKPROPAGATION

- **Gradient Descent:** $\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}$, $\eta \equiv \text{learning rate}$.
- **Mini-batch SGD:** Promedia $\nabla \mathcal{L}$ en un subconjunto aleatorio (batch). Suaviza la trayectoria, paraleliza matrices en GPU y evade mínimos locales.
- **Backpropagation:** Diferenciación algorítmica. El error $\delta^{(l)}$ de la capa l se calcula retropropagando el error de la capa posterior $l+1$:
$$\delta^{(l)} = \left((W^{(l+1)})^T \delta^{(l+1)} \right) \odot f'(z^{(l)}) \implies \nabla_{W^{(l)}} \mathcal{L} = \delta^{(l)} (\mathbf{a}^{(l-1)})^T, \quad \nabla_{b^{(l)}} \mathcal{L} = \delta^{(l)}$$
- **Problema de Alta Dimensión:** Los puntos de silla ($\nabla \mathcal{L} = 0$ con curvatura mixta) son más probables que los mínimos locales. El SGD requiere inercia (*momentum*) para escapar.
- **Adam (Adaptive Moment Estimation):** Mejora el SGD al combinar *momentum* con una tasa de aprendizaje dinámica e independiente para cada parámetro. Esto estabiliza las oscilaciones y acelera la convergencia en topologías complejas de alta dimensión.

IV. REGULARIZACIÓN Y PCA

COMPROMISO SESGO-VARIANZA

- Error = Bias² + Var + Ruido. Regularizar aumenta el sesgo para hundir la varianza, alcanzando el error de generalización mínimo.
- **Underfitting (Sesgo):** Modelo rígido. Error train alto.
 - **Overfitting (Varianza):** Memoria ruido. Divergencia $\mathcal{L}_{train} \rightarrow 0$, $\mathcal{L}_{test} \gg 0$.

REGULARIZACIÓN GEOMÉTRICA (Ω)

Restricción del espacio de fases de los pesos $\mathcal{L}_{reg} = \mathcal{L} + \alpha \Omega(w)$.

- **L2 (Ridge):** $\Omega(w) = \|w\|_2^2$. Contrae coeficientes proporcionalmente (suaviza). Espacio de restricción: **Esfera**. Maneja bien señales distribuidas y multicolinealidad.
- **L1 (Lasso):** $\Omega(w) = \|w\|_1$. Induce Sparsity (fuerza pesos a exactamente 0). Espacio de restricción: **Rombo/Diamante**. Las curvas de error suelen colisionar en los vértices del rombo, anulando ejes (selección de variables automática).
- **Elastic Net:** Combinación $\alpha_1 \|w\|_1 + \alpha_2 \|w\|_2^2$.

TÉCNICAS ESTRUCTURALES

- **Dropout:** Probabilidad p de silenciar neuronas en *forward*. Simula un ensamblado de 2^N sub-redes. Evita la co-adaptación de neuronas. Convergencia lenta.
- **Early Stopping:** Detiene optimización si $\mathcal{L}_{val}$ se satura o diverge.

DATA LEAKAGE (FUGA DE INFORMACIÓN)

El peor error en ML. Contaminar el entrenamiento con estadísticas del test.

- **Pecados:**
 - 1) Normalizar usando μ, σ globales antes del split.
 - 2) Seleccionar features o PCA globalmente.
 - 3) Tuning abusivo mirando el test set.
 - 4) Filas correlacionadas físicamente $\Rightarrow$ Solución: GroupShuffleSplit o GroupKFold.

PROTOCOLOS DE PARTICIÓN

- **Train Set (60-80 %):** Optimización exclusiva de los tensores paramétricos.
- **Validation Set (10-20 %):** Ajuste de hiperparámetros (η, α, p). Proxy del test.
- **Test Set (10-20 %):** Evaluación de generalización única y final.
- **K-Fold CV:** Partición en K pliegues iterativos. Reduce la varianza en la estimación del rendimiento para datasets reducidos.
- **Group Split:** Imprescindible si los datos físicos no son i.i.d. Agrupa por identificador de muestra para asegurar independencia total.

MÉTRICAS DE CLASIFICACIÓN Y MATRIZ DE CONFUSIÓN

- El error 0/1 no es diferenciable, se optimiza NLL pero se reportan métricas duras.
- Aciertos: True Positive/Negative (TP, TN). Errores: False Positive/Negative (FP, FN).
- **Matriz:** Predición vs Actual value. Aciertos en la diagonal principal.
 - **Accuracy:** $(TP + TN) / Total$. Inválida en *Imbalanced Data*.
 - **Recall (Sensibilidad):** $TP / (TP + FN)$. De los positivos reales, ¿cuántos cazo?
 - **Especificidad:** $TN / (TN + FP)$. De los negativos reales, ¿cuántos descarto?
 - **Precisión:** $TP / (TP + FP)$. Probabilidad de que mi alerta positiva sea real.
 - **ROC-AUC:** Área bajo la curva (AUC) de ROC (sensibilidad frente a 1-especificidad). Mide la capacidad de separar clases. AUC = 1: Clasificador perfecto.

V. MÉTODOS ESTADÍSTICOS Y PCA

PRINCIPAL COMPONENT ANALYSIS (PCA)

- Transformación lineal ortogonal no supervisada que proyecta $X \rightarrow Z = XW_k$. Reduce multicolinealidad, filtra ruido y permite visualización.
- **Preprocesado Crítico:** Centrar $\mu = 0$ y Escalar $\sigma = 1$ (StandardScaler).
 - **Covarianza y Autovectores:** La matriz de pesos de proyección W_k está formada por los k autovectores principales de la matriz de covarianza $\Sigma = \frac{1}{N} X^T X$.

- **Cálculo SVD:** $X = USV^T$. Computacionalmente estable. Los ejes principales son las columnas de V . Los autovalores λ_j relacionan los valores singulares: $\lambda_j = \sigma_j^2 / N$.
- **Varianza Explicada (r_j):** Fracción de variabilidad retenida $r_j = \lambda_j / \sum \lambda_i$. Se elige k truncando la varianza acumulada al 90-95 %.

k-NEAREST NEIGHBORS (k-NN)

- Algoritmo no paramétrico (*lazy learning*) con coste de inferencia $\mathcal{O}(nd)$. Requiere estructuras espaciales (KD-tree, Ball-tree) para acelerar.
- **Distancias (L_p):** Minkowski, Euclídea, ... Exige estandarización dimensional.
 - **Altas dimensiones:** Las distancias colapsan y la métrica pierde su poder discriminativo. Requiere PCA previo.
 - **Geometría y Sesgo-Varianza:** $k = 1$ (Altamente no lineal, varianza máxima). k grande (Fronteras de decisión promediadas, máximo sesgo predictivo).

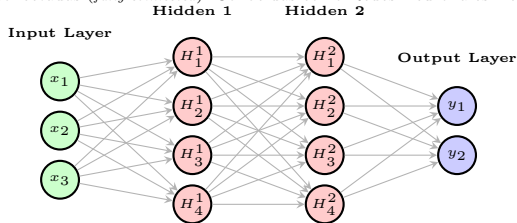
VI. ARQUITECTURAS NEURONALES AVANZADAS

1. EL PERCEPTRÓN Y LA LÓGICA

El perceptrón divide el espacio con un hiperplano $w^T \mathbf{x} + b = 0$.

2. MULTILAYER PERCEPTRON (MLP)

Arquitectura *feedforward* (flujo unidireccional de información) compuesta de capas totalmente conectadas (*fully connected*). Conocidas como Redes Neuronales Densas.

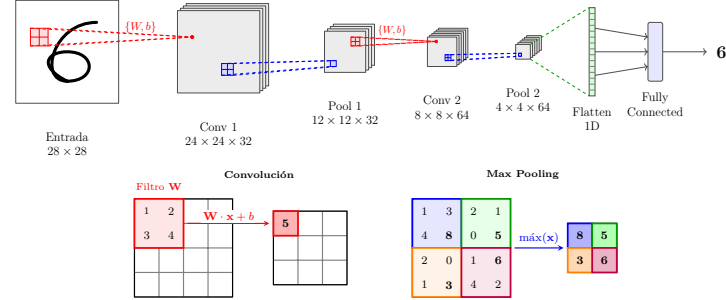


- **Teorema de Aproximación Universal:** Una ANN (*artificial neural network*) con 1 capa oculta (ancho ilimitado) y activación no polinómica puede mapear con precisión arbitraria cualquier función continua $\phi : \mathbb{R}^n \rightarrow \mathbb{R}^m$.

$$\mathbf{h} = f(W^{(1)} \mathbf{x} + b^{(1)}) \rightarrow \hat{y} = g(W^{(2)} \mathbf{h} + b^{(2)})$$

3. CONVOLUTIONAL NEURAL NETWORKS (CNN)

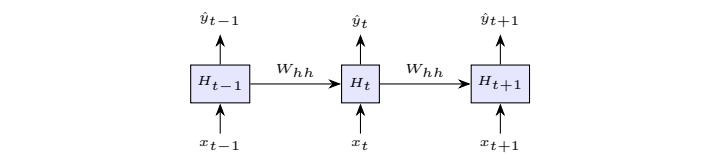
Arquitecturas diseñadas para invarianzas topológicas en mallas de datos (Grid).



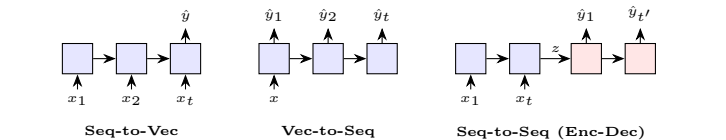
- **Convolución:** Operador lineal local que desliza un filtro $\mathbf{W} \in \mathbb{R}^{f \times f \times C_{in}}$ sobre un tensor $\mathbf{X}$. Compuesto por C_{in} kernels $\mathbf{K}$ (uno por canal). Impone localidad y estacionariedad (pesos compartidos), reduciendo la complejidad paramétrica.
- **Pooling (submuestreo determinista):** Operador de proyección espacial sin parámetros aprendibles. Ejemplos: Max Pooling, Average Pooling, ...
- **Dimensionalidad:** Para dimensión de entrada W_{in} , el tamaño del kernel f , padding p (completado periférico) y stride s (paso de desplazamiento):
$$W_{out} = \lfloor \frac{W_{in} - f + 2p}{s} \rfloor + 1$$

6. RECURRENT NEURAL NETWORKS (RNN)

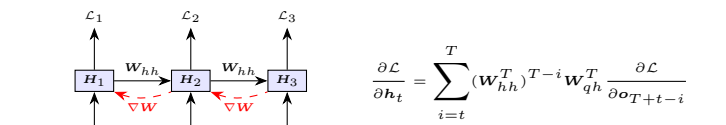
Topologías con inercia para series temporales y procesamiento secuencial (NLP). Proyectan $\mathbf{x}_t$ y un vector de memoria interna $\mathbf{h}_t$.



- **Topologías Típicas:** Seq-to-vec (Clasificación/Forecasting de series temporales), Vec-to-seq (Generación de texto), Seq-to-seq (Traducción).



- **Backpropagation Through Time (BPTT):** El gradiente temporal se desenrolla, mostrando inestabilidad intrínseca por la propagación encadenada sobre W_{hh} .



Problema: Las potencias sucesivas de matrices W_{hh} provocan inestabilidad de Liapunov.

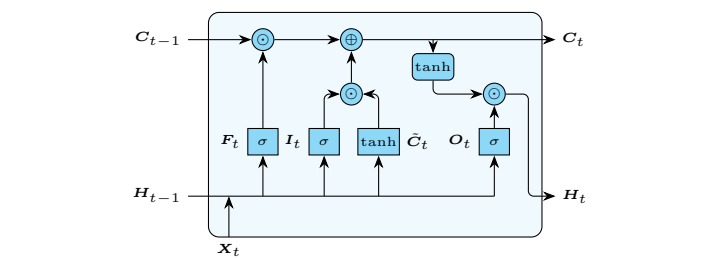
Si $\lambda_{max} > 1$, la explosión del gradiente destruye la red. Se realiza un recorte radial mediante **Gradient Clipping**: $\mathbf{g} \leftarrow \min(1, \theta / \|\mathbf{g}\|) \cdot \mathbf{g}$

Si $\lambda_{max} < 1$, el *Vanishing Gradient* causa amnesia en el tiempo. Requiere **Gating**.

LONG SHORT-TERM MEMORY (LSTM) & GATED RECURRENT UNIT (GRU)

Arquitecturas Gated. Aseguran estabilidad del gradiente mediante operadores de saturación ($\sigma \in [0, 1]$) que regulan el transporte aditivo de información, impidiendo la contracción exponencial de la norma en el espacio de estados durante BPTT.

- **LSTM:** Desacopla la memoria latente (C_t) del estado observable (H_t), permitiendo una regulación asíncrona de la persistencia mediante una puerta de olvido dedicada.



Ecuaciones de LSTM: (Interacción de Estado Oculto H_t y Celda de Estado C_t).

$$F_t = \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f) \quad (\text{Forget Gate})$$

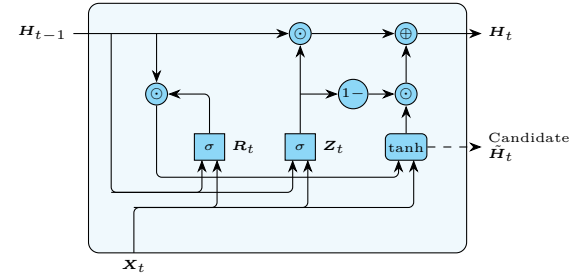
$$I_t = \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i) \quad (\text{Input Gate})$$

$$C_t = F_t \odot C_{t-1} + I_t \odot \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c) \quad (\text{Celda})$$

$$O_t = \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o) \quad (\text{Output Gate})$$

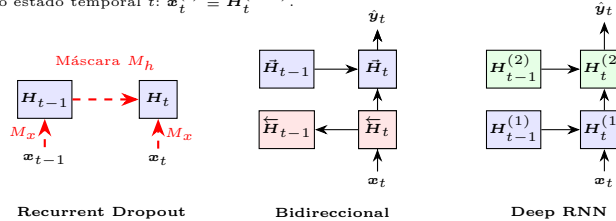
$$H_t = O_t \odot \tanh(C_t) \quad (\text{Hidden State})$$

• **GRU:** Reducción dimensional del LSTM. Fusiona la celda y elimina la compuerta de salida, operando directamente sobre tensores de **Reset** (R_t) y **Update** (Z_t):

$$H_t = Z_t \odot H_{t-1} + (1 - Z_t) \odot \tilde{H}_t$$


TOPOLOGÍAS RECURRENTES AVANZADAS

- **Recurrent Dropout:** Previene el sobreajuste preservando el flujo continuo de información de BPTT. Fija estocásticamente la misma máscara binaria de anulación para todos los timesteps t sobre el tensor recurrente.
- **RNNs Bidireccionales:** Procesamiento acausal de secuencias. Acopla una barrida cronológica ($\tilde{H}_t$) y otra anticronológica ($\bar{H}_t$), resultando en $H_t = [\tilde{H}_t; \bar{H}_t] \in \mathbb{R}^{n \times 2h}$.
- **Deep RNNs (Stacked):** Apilamiento espacial de la memoria temporal. La dinámica oculta de la capa inferior actúa como vector de entrada para la capa superior dentro del mismo estado temporal t : $x_t^{(l)} \equiv H_t^{(l-1)}$.



VII. NLP: PREPROCESAMIENTO Y TOKENIZACIÓN

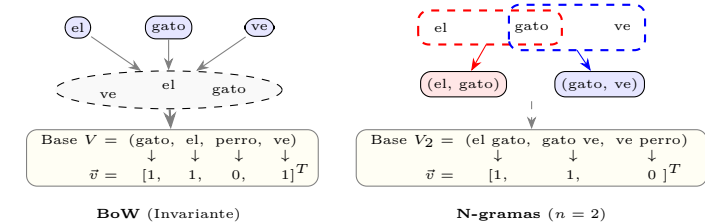
PIPELINE DE TEXTO (REDUCCIÓN DEL ESPACIO DE ESTADOS)

- **Estandarización:** Eliminación de acentos, mayúsculas y ruido sintáctico para reducir la dimensionalidad cruda del espacio léxico antes de la vectorización.
 - **Estrategias de partición (Tokenización):**
- Caracteres:** Vocabulario mínimo ($|V| \sim 80$). Problema: Las secuencias se alargan demasiado, haciendo inviable el cómputo de la atención.
- Palabras:** Secuencias cortas. Problema: El vocabulario explota ($|V| \rightarrow \infty$). Falla con palabras no vistas (OOV), exigiendo el uso del token destructivo [UNK].
- Subpalabras o Tokens:** Se obtienen mediante BPE (Byte-Pair Encoding), algoritmo greedy que fusiona iterativamente los pares de bytes/caracteres más frecuentes. Elimina la necesidad de [UNK] y acota la longitud de los tensores.

- **Tokens de control:** Delimitadores lógicos como [START] y [END]. El token [PAD] actúa como relleno escalar para homogeneizar dimensiones, permitiendo apilar arrays en matrices densas para *batching*.

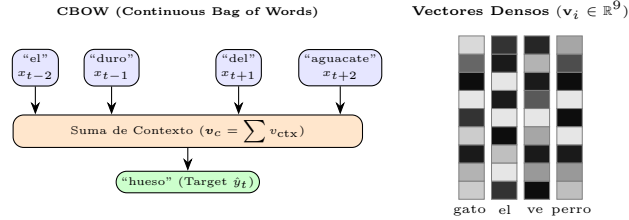
FASE 1: REPRESENTACIÓN DISCRETA (MODELOS BASADOS EN SETS)

- **Bag-of-Words (BoW):** Proyección *multi-hot* en $\mathbb{R}^{|V|}$ (vector con 1 en índices de todas palabras presentes en V , 0 elsewhere). Genera matrices extremadamente dispersas (*sparse*). Es un modelo invariante ante permutaciones; destruye por completo el orden secuencial de la frase.
- **N-gramas:** Captura el orden local tomando ventanas de radio n . La frecuencia de aparición de los n -gramas decae bruscamente siguiendo la Ley de Zipf ($f \propto 1/k$). Genera matrices de transición inestables por escasez de datos (maldición de la dimensionalidad).



FASE 2: ESPACIOS CONTINUOS (WORD EMBEDDINGS)

- **CBOW:** Arquitectura algorítmica utilizada empíricamente para pre-entrenar la matriz de embedding W . Desliza una ventana continua sobre todo el texto del dataset, intentando adivinar la palabra central que falta basándose en el contexto directo a su izquierda y derecha. Al forzar a la red a predecir repetidamente el target $y_t \in V$ usando el promedio de las proyecciones $w_{t \pm i}$, los tensores convergen iterativamente hacia un rico hiperespacio semántico de forma autosupervisada.
- **Embedding:** Mapeo paramétrico que proyecta un vocabulario discreto a un espacio denso continuo: $|V| \rightarrow \mathbb{R}^d$ (dimensión de la matriz de embedding: $W \in \mathbb{R}^{V \times d}$). La métrica de similitud colapsa en el producto interno vectorial, erradicando la ortogonalidad estricta del BoW (*obsérvese la similitud entre gato y perro*). Limitación: El mapeo es estático y biunívoco; independiente de su posición temporal en una nueva frase.

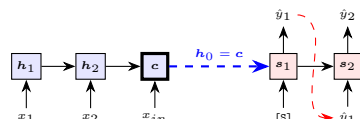


FASE 3: OPERADOR DINÁMICO TEMPORAL (RNN Seq2Seq)

El modelo Sequence-to-Sequence (Seq2Seq) establece un mapeo asimétrico entre una serie temporal de entrada y otra de salida ($T_{in} \neq T_{out}$). Opera mediante el acoplamiento de un Encoder, que comprime la información, y un Decoder, que la expande.

- **Compresión del Espacio Latente (Encoder):** Lee la serie fuente $\mathbf{x} = (x_1, \dots, x_{T_{in}})$ de manera secuencial. En cada instante temporal t , el estado oculto h_t se actualiza integrando el token actual y la memoria del estado previo: $h_t = f(x_t, h_{t-1})$. Una vez asimilada la secuencia completa, la serie de datos colapsa en un vector de contexto singular $\mathbf{c} = h_{T_{in}}$. Este vector asume la responsabilidad matemática de representar toda la semántica y sintaxis de $\mathbf{x}$.
- **Inferencia Estocástica Autoregresiva (Decoder):** Generador estocástico, donde la probabilidad conjunta de la trayectoria se descompone asumiendo que el futuro no influye en el presente: $P(\hat{y}_1 \dots \hat{y}_{T_{out}}) = \prod P(\hat{y}_t | \hat{y}_{<t}, \mathbf{c})$. El procedimiento es el siguiente:

1. Utiliza el vector $\mathbf{c}$ como condición inicial ($s_0 = \mathbf{c}$) para un nuevo proceso markoviano.
2. En cada instante t , el sistema acopla el token generado en el paso anterior con la memoria actual para actualizar el estado del sistema (s_t).
3. El sistema proyecta hiperspacialmente su estado continuo hacia la dimensionalidad del vocabulario para obtener la probabilidad del siguiente token: $P(\hat{y}_t) = \text{Softmax}(W_{ho} s_{t-1})$.
4. Se extrae el token observable $\hat{y}_t$ (mediante la máxima probabilidad o muestreo sobre $P(\hat{y}_t)$). El token predicho $\hat{y}_t$ se retroinyecta inmediatamente en el sistema como entrada para calcular el instante $t+1$, formando un bucle cerrado.



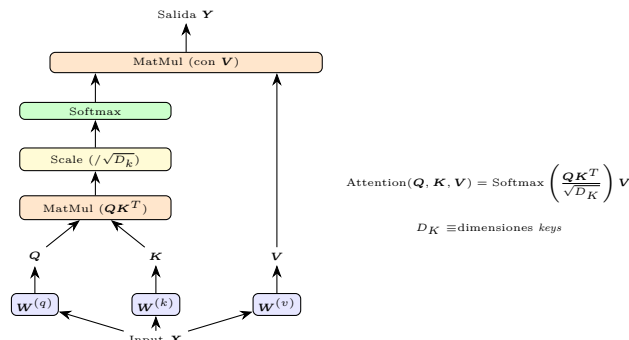
• LIMITACIONES DEL Seq2Seq:

- **Bottleneck:** Toda la información de la secuencia de entrada colapsa en el vector $\mathbf{c} \in \mathbb{R}^d$. A medida que T_{in} crece, la entropía de la secuencia excede la capacidad de representación de un vector de dimensión fija y forzando al modelo a comprimir con pérdidas.
- **Amnesia Temporal:** Debido a la multiplicación recursiva de matrices de peso en la RNN a través del tiempo, los gradientes tienden a desvanecerse, atenuando exponencialmente la influencia del pasado distante.

VIII. TRANSFORMERS Y MECANISMO DE ATENCIÓN

SCALED DOT-PRODUCT SELF-ATTENTION

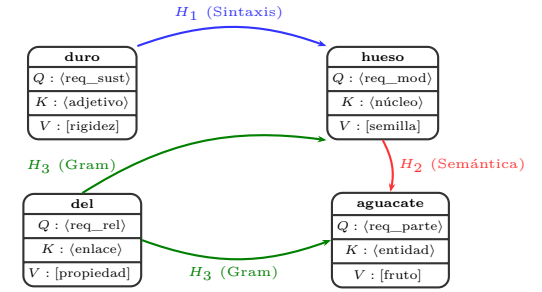
Sustituye la recurrencia (aunque comenzó como Seq2Seq con pesos a las h_i que crean $\mathbf{c}$). Compara un vector de consulta (*Query*, Q) con claves (*Keys*, K) para asignar pesos a los valores (*Values*, V). Término $1/\sqrt{D_K}$ previene saturación en Softmax y gradientes nulos.



- **Multi-Head Attention:** Paraleliza H proyecciones lineales independientes para capturar subespacios ortogonales correlacionados:

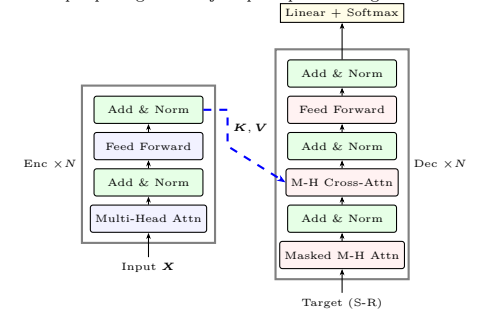
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$



LA TOPOLOGÍA SISTÉMICA DEL TRANSFORMER

Bloques acoplados de *multi-head attention* mediante normalización de capa (*LayerNorm*) y conexiones residuales que protegen el flujo hiperspacial del gradiente.



POSITIONAL ENCODING Y ROPE

Al procesar conjuntos (sets) de vectores de forma paralela $\mathcal{O}(1)$, el orden topológico secuencial debe inyectarse explícitamente.

- **Sinusoidal:** Suma estática al embedding base: $\tilde{\mathbf{x}}_n = \mathbf{x}_n + \text{PE}(n) \equiv \text{positional encoding}$.
- **Positional Embedding Aprendizido:** Se aprende una matriz de pesos W_{pos} , pero no generaliza bien a secuencias más largas que las vistas en el entrenamiento y no es invariante ante permutaciones (PE sí lo es).

IX. TRANSFORMERS: CROSS-ATT., GPT Y BERT

1. CROSS-ATTENTION (SEQ2SEQ)

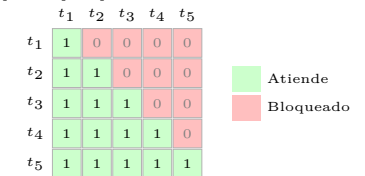
El Decoder necesita consultar la salida del Encoder Z . La *cross-attention* es idéntica a la self-attention salvo que Q proviene del Decoder mientras que K, V provienen de Z :

$$\text{CrossAttn}(Q_{dec}, K_{enc}, V_{enc}) = \text{Softmax}\left(\frac{Q_{dec} K_{enc}^T}{\sqrt{D_K}}\right) V_{enc}$$

Esto permite al Decoder “consultar” libremente cualquier posición de la secuencia fuente en cada paso generativo, resolviendo el bottleneck del Seq2Seq clásico.

CAUSAL MASK (TRANSFORMER DECODER)

La atención es inherentemente bidireccional. Para generación autoregresiva se aplica máscara triangular *upper* que bloquea posiciones futuras: el token i solo atiende $j \leq i$.



`torch.triu(..., diagonal=1) -> True = bloqueado`

2. GPT: DECODER-ONLY TRANSFORMER

Modelo autoregresivo puro: solo Decoder con masked self-attention. Aprende sin supervisión $P(x_n | x_1, \dots, x_{n-1})$ sobre cualquier texto plano. Sin encoder, no puede hacer cross-attention, por lo que combina contexto y respuesta en una **única secuencia**.

– **Ventaja:** Escalable a la totalidad de Internet sin curación de pares.

– **Limitación:** Flujo unidireccional incluso para el contexto de entrada (vs. BERT).

Warmup Schedule: Para evitar explosión del gradiente al inicializar un Transformer profundo, el learning rate se incrementa linealmente durante $N_{warm\ up}$ pasos antes de mantenerse constante (o decaer):

$$\eta_t = \eta_{\max} \cdot \min\left(\frac{t}{N_{warm\ up}}, 1\right)$$

Key-Value Caching: Técnica de inferencia que almacena los tensores K, V calculados en pasos anteriores, evitando recomputarlos en cada nuevo token generado. Reduce el costo de generación de $\mathcal{O}(T^2)$ a $\mathcal{O}(T)$ por token.

ESTRATEGIAS DE MUESTREO (SAMPLING): Dado $P(\hat{y}_t | \hat{y}_{<t}) \in \mathbb{R}^{|V|}$, hay varias estrategias:

- **Greedy:** $\hat{y}_t = \arg \max P$. Determinista, rápido, puede quedar en soluciones subóptimas.
- **Beam Search** (ancho B): Mantiene las B hipótesis de mayor probabilidad acumulada. Normaliza la prob. por longitud para evitar sesgo hacia secuencias cortas: $\log P/|y|$.
- **Temperatura T :** Reescala los logits antes del Softmax: $y_i = \exp(a_i/T) / \sum_j \exp(a_j/T)$

$T \rightarrow 0$: greedy. $T = 1$: Softmax estándar. $T \gg 1$: distribución uniforme.

- **Top-K:** Muestra solo entre los K tokens de mayor probabilidad (trunca la distribución). Beam Search + Top-K + $T < 1$ es la combinación estándar en producción.

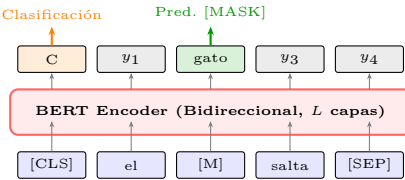
3. BERT: ENCODER TRANSFORMER

Pre-entrenamiento bidireccional sobre corpus masivo, luego fine-tuning supervisado para tareas downstream con pocos datos (*transfer learning*).

Tareas de pre-entrenamiento (autosupervisadas):

- **MLM (Masked Language Modeling):** 15% de tokens se enmascaran. La red predice el token original usando **contexto bidireccional** (izquierda y derecha). Análogo a CBOW pero a nivel profundo de representación.
- **NSP (Next Sentence Prediction):** Clasifica si la frase B sigue naturalmente a la frase A. Captura coherencia discursiva.

Fine-tuning: Se añade una cabeza (*head*) de salida específica a la tarea. El token especial [CLS] acumula la representación global de la secuencia para clasificación.



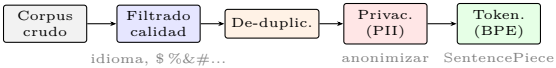
GPT vs BERT: Resumen comparativo

	GPT	BERT
Arquitectura	Decoder-only	Encoder-only
Atención	Unidireccional	Bidireccional
Entrenamiento	Next-token pred.	MLM + NSP
Uso ideal	Generación	Clasificación / NER
Ejemplos	ChatGPT, GPT-4	RoBERTa, SciBERT

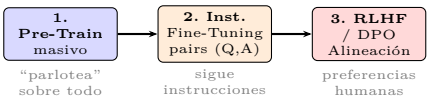
NER: Named Entity Recognition [Euler nació en Suiza → Euler: persona, Suiza: lugar]

X. LLMS: ENTRENAMIENTO Y FINE-TUNING

PIPELINE DE PRE-ENTRENAMIENTO
Preparación del corpus antes de entrenar el LLM fundacional:



POST-TRAINING: 3 ETAPAS DE ALINEACIÓN



INSTRUCTION FINE-TUNING
Se entrena sobre pares (instrucción, respuesta) multi-tarea. Generaliza a tareas no vistas. Variante: **Chain-of-Thought FT** → incluye razonamiento paso a paso en la respuesta.

PARAMETER-EFFICIENT FINE-TUNING (PEFT)
Fine-tuning completo de LLMs (>10B params): inviable en hardware convencional porque las redes son sobreparametrizadas: las actualizaciones ΔW tienen *rango intrínseco bajo*. PEFT actualiza solo un subconjunto de parámetros $\ll$ parámetros totales (eg. LoRA).

RLHF (Reinforcement Learning from Human Feedback)
Pipeline de 3 pasos para alinear el LLM con preferencias humanas:

1. **SFT:** Fine-tuning supervisado sobre demostraciones de alta calidad.
2. **Reward Model r_θ :** Red entrenada sobre comparaciones humanas entre pares de respuestas para el mismo prompt (ranking relativo / ELO).
3. **Optimización RL con PPO:** El LLM (política π) maximiza la recompensa esperada con penalización KL para no alejarse del SFT:

- **DPO (Direct Preference Optimization):** Elimina el modelo de recompensa explícito. Optimiza directamente sobre pares de preferencias con una pérdida de clasificación binaria, extrayendo la política óptima en forma cerrada. El LLM es implícitamente el modelo de recompensa. Más simple y estable que RLHF.

PROPIEDADES EMERGENTES Y TÉCNICAS DE PROMPTING

- **Zero-Shot:** Solo instrucción textual. Requiere modelos grandes con instruction-tuning.
- **Few-Shot:** k ejemplos (Q, A) como demostración implícita de formato y tarea.
- **CoT Prompting:** Few-shot con razonamiento explícito: "First, X. Then, Y. Therefore, Z."
- **RAG (Retrieval-Augmented Generation):** Recupera documentos relevantes de una base de datos vectorial (*embedding search*) para enriquecer el contexto antes de inferencia. Mitiga alucinaciones y el límite de conocimiento temporal (*knowledge cutoff*).

XI. GENERATIVE MODELS: GAN y VAE

BIG PICTURE GENERATIVO
A diferencia del aprendizaje discriminativo, los modelos generativos aprenden y modelan la topología estadística del *espacio latente*. Permiten mostrar trayectorias en este subespacio continuo para proyectar nuevos tensores artificiales que respeten la distribución original p_{data} .

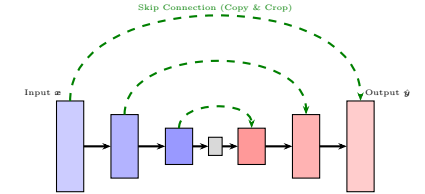
1. GENERATIVE ADVERSARIAL NETWORKS (GAN)
Aproximación estocástica basada en un juego minimax de suma cero. Dos redes neuronales acopladas compiten hacia un Equilibrio de Nash:

- **Discriminador (D):** Clasificador binario $D: \mathbb{R}^n \rightarrow (0, 1)$. Maximiza la probabilidad de separar datos reales ($\mathbf{x}$) de ficticios ($G(\mathbf{z})$). Equivalente a minimizar la BCE.
- **Generador (G):** Mapea ruido latente $\mathbf{z} \sim \mathcal{N}(0, I)$ (discreto) al espacio físico $\hat{\mathbf{x}} = G(\mathbf{z})$. Su objetivo es maximizar la tasa de fallo del Discriminador.

$$\min_D \max_G \mathcal{V}(D, G) = \mathbb{E}_{\mathbf{x}} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}} [\log (1 - D(G(\mathbf{z})))]$$

UPSAMPLING Y ARQUITECTURA ESPACIAL
Para transformar el escalon latente $\mathbf{z}$ en mallas espaciales (imágenes), el Generador sustituye el *pooling* convencional por operadores de expansión métrica:

- **Transpose Convolution:** Proyecta escalares a parches matriciales deslizando un filtro. Soluciona los artefactos de tablero de ajedrez (*checkerboard*) si el tamaño del *kernel* es múltiplo exacto del *stride*.
- **Max Unpooling:** Revierte el submuestreo de forma determinista, recolocando los tensores en los índices espaciales originales memorizados durante el *MaxPool*.
- **U-Net (Segmentación):** Red *Encoder-Decoder* simétrica. Integra conexiones residuales (*Copy and Crop*) que acoplan mapas de características del encoder directamente al decoder, preservando la información espacial de alta frecuencia que fue destruida en el colapso latente.

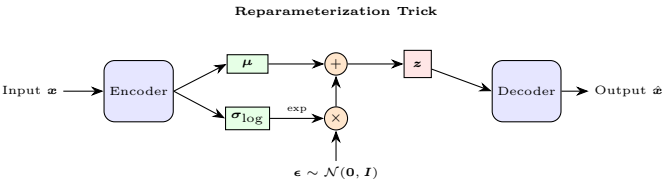


TOPOLOGÍA DE ENTRENAMIENTO (DCGAN)
Requiere inducción de sesgos arquitectónicos muy rígidos para converger:

- **Activaciones:** Generador remata con Tanh (fuerzando output $\in [-1, 1]$). Las capas ocultas exigen LeakyReLU ($\approx 1E-2 \cdot x$ si $x < 0$) en lugar de ReLU, previniendo el colapso del flujo del gradiente (*sparse gradients*).
- **Batch Normalization:** Obligatorio. Mitiga el *Internal Covariate Shift* forzando $\mu \approx 0, \sigma \approx 1$ localmente en los minibatches, estabilizando la dinámica de pesos.
- **Optimizador Adam:** Crítico restringir la inercia. Usar $\beta_1 = 0.5$ (frente al 0.9 estándar) acorta la memoria del gradiente de primer orden, frenando oscilaciones divergentes sobre el óptimo dinámico.
- **Estocasticidad:** Inyectar ruido gaussiano en $\mathbf{z}$, usar Dropout en D o introducir ruido en las etiquetas previene el sobreajuste rápido de D que aniquilaría la actualización de G .

2. VARIATIONAL AUTOENCODERS (VAE)
Un Autoencoder determinista colapsa $\mathbf{x}$ en un punto $\mathbf{z}$ arbitrario (generando un espacio latente fragmentado y discontinuo). Un VAE impone restricciones topológicas bayesianas, forzando a que la codificación describa los estadísticos de una distribución normal multivariada (μ, σ).

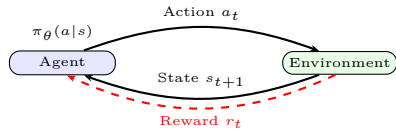
- **Reparameterization Trick:** Para que el grafo sea puramente diferenciable, el muestreo estocástico se externaliza a un nodo de ruido residual $\epsilon \sim \mathcal{N}(0, I)$:
$$\mathbf{z} = \mu + \exp(\sigma \log) \odot \epsilon$$
- **Superficie de Pérdida:** Minimiza la *Reconstruction Loss* (fidelidad predictiva) penalizada por *Regularization Loss* (reduce *overfitting* + espacios latentes compactos).
- **Vectores de Concepto:** Al garantizar un hiperespacio topológicamente continuo y densamente estructurado, la traslación vectorial $\mathbf{z}' = \mathbf{z} + \mathbf{v}$ induce cambios directos en propiedades físicas o semánticas aisladas (ej. la transición de una cara seria a una sonriente).



	GAN	VAE
Dinámica	Eq. Nash (Minimax)	Inferencia Bayesiana (ELBO)
Espacio Latente	Libre $\mathbf{z} \sim \mathcal{N}(0, I)$	Construido a $\mathcal{N}(\mu, \sigma)$
Pérdida ($\mathcal{L}$)	BCE comp. $\mathcal{V}(D, G)$	Divergencia $D_{KL} + \mathcal{L}_{recon}$
Muestreo	Nítido (riesgo <i>mode collapse</i>)	Cobertura total (suavizado/ <i>blurring</i>)
Interpolación	Inestable (vacíos topológicos)	Continua (vectores de concepto)

XII. REINFORCEMENT LEARNING (RL)

DEFINICIÓN Y ENTIDADES
A diferencia del aprendizaje supervisado, el RL no requiere etiquetas, sino una señal escalar de retroalimentación (**Reward**). El agente aprende una **política** π optimizando la toma de decisiones secuenciales por ensayo y error.



MARKOV DECISION PROCESS (MDP)
Formalismo matemático $\{S, A, T, R, \gamma\}$:

- **State Space (S):** Configuración actual del entorno.
- **Action Space (A):** Conjunto de movimientos posibles.
- **Transition (T):** $P(s_{t+1} | s_t, a_t)$. Dinámica del sistema. Si es desconocida → *Model-free*.
- **Reward (R):** Señal inmediata $R(s_t, a_t, s_{t+1})$.
- **Discount ($\gamma \in [0, 1]$):** Prioriza rewards inmediatas frente a futuras. Evita sumas ∞ .

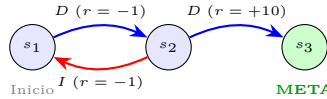
CONCEPTOS CLAVE DE TRAYECTORIA

- **Trajectory (τ):** Secuencia $(s_0, a_0, r_0, s_1, a_1, \dots)$.
- **Return (G_t):** Suma de recompensas descontadas: $G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}$.
- **Objective (J):** Maximizar la esperanza del retorno: $J(\pi) = \mathbb{E}_{\tau \sim \pi} [G(\tau)]$.

ESTIMACIÓN DEL RENDIMIENTO

- **State-Value $V^\pi(s)$:** ¿Cómo de bueno es estar en el estado s ?
- **Action-Value $Q^\pi(s, a)$:** ¿Cómo de bueno es tomar la acción a en el estado s ?

EJEMPLO CLÁSICO: EL CORREDOR DISCRETO (GRIDWORLD 1D)
Entorno lineal de 3 celdas discretas donde el agente busca alcanzar una celda meta (s_3).



- $\{S, A, T, R, \gamma\}$
- **State Space (S):** Tres posiciones posibles. $S = \{s_1, s_2, s_3\}$.
- **Action Space (A):** Movimientos binarios. $A = \{\text{Derecha } (D), \text{Izquierda } (I)\}$.
- **Transition (T):** Determinista, probabilidad de éxito es 1. Ej: $P(s_2 | s_1, D) = 1$.

Si fuese *slippery*, el "hielo" haría que el movimiento sea incierto. Ej: $P(s_2 | s_1, D) = 0.8$, $P(s_1, s_1, D) = 0.2$: pese a "querer" ir a la derecha, se cae y se queda en la misma celda.

- **Reward (R):** Penalización por coste temporal o energético $r = -1$ en transiciones ordinarias; incentivo $r = +10$ únicamente al entrar en la meta s_3 .
- **Discount (γ):** Fijamos $\gamma = 0.9$ para ponderar el horizonte temporal.

● **Trajectory y rendimiento**
Evaluamos una política determinista hacia la derecha: $\pi(s_1) = D$ y $\pi(s_2) = D$.

- **Trajectory (τ):** Partiendo desde s_1 , la secuencia exacta de estados, acciones y recompensas hasta el colapso en el estado terminal es: $\tau = (s_1, D, -1, s_2, D, +10, s_3)$
- **Return (G_t):** El retorno acumulado descontado desde el estado inicial ($t = 0$) deshace la recursividad temporal: $G_0 = r_0 + \gamma r_1 = -1 + 0.9 \cdot (+10) = 8.0$
- **State-Value $V^\pi(s)$:** Mide la esperanza del retorno desde cada estado. Sabiendo que s_3 es terminal y su valor futuro es nulo ($V(s_3) = 0$):
$$V^\pi(s_2) = R(s_2, D, s_3) + \gamma V^\pi(s_3) = +10 + 0.9 \cdot 0 = 10.0$$
$$V^\pi(s_1) = R(s_1, D, s_2) + \gamma V^\pi(s_2) = -1 + 0.9 \cdot 10.0 = 8.0$$
- **Action-Value $Q^\pi(s, a)$:** Evalúa el impacto de tomar una acción potencialmente errónea antes de retomar la política original π . En el estado s_2 :
$$Q^\pi(s_2, D) = R(s_2, D, s_3) + \gamma V^\pi(s_3) = +10 + 0 = 10.0$$
$$Q^\pi(s_2, I) = R(s_2, I, s_1) + \gamma V^\pi(s_1) = -1 + 0.9 \cdot 8.0 = 6.2$$

Como $Q^\pi(s_2, D) > Q^\pi(s_2, I)$, el operador de optimalidad de Bellman seleccionará la acción D , consolidando el sesgo de la política óptima π^* .

TAXONOMÍA DE ALGORITMOS RL

1. VALUE-BASED (Q-LEARNING y SARSA)
Aprenden el valor de las acciones. La política es implícita (ϵ -greedy).

- **Temporal Difference (TD) Learning:** Actualiza la estimación basada en la diferencia entre el valor actual y el valor observado + siguiente (*TD-Error*).
- **SARSA (On-Policy):** Aprende de la acción realmente tomada por la política.
- **Q-Learning (Off-Policy):** Aprende de la **mejor** acción posible en el siguiente estado.
$$Q_{new}^{SARSA} = Q + \alpha[r + \gamma Q(s', a') - Q]$$
$$Q_{new}^{Q-Learn.} = Q + \alpha[r + \gamma \max_{a'} Q(s', a') - Q]$$

2. POLICY-BASED (REINFORCE)
Optimiza directamente $\pi_\theta(a|s)$ mediante ascenso de gradiente.

- **Policy Gradient Theorem:** $\nabla_\theta J(\theta) = \mathbb{E}[G_t \nabla_\theta \ln \pi_\theta(a_t | s_t)]$.
- **Ventaja:** Maneja espacios de acciones continuos y políticas estocásticas.

3. COMBINED (ACTOR-CRITIC Y PROXIMAL POLICY OPTIMIZATION)

- **Actor:** Aprende la política (qué hacer).
- **Critic:** Aprende la función de valor (evalúa al actor).
- **Advantage (A):** $A(s, a) = Q(s, a) - V(s)$. Indica cuánto mejor es una acción respecto al promedio.